

Relatório de Benchmark — SGBDs Relacionais

Resumo interno para entender o que foi testado, o que esperávamos e o que os dados mostraram.
Não é o relatório final.

1. O que estamos analisando

Três bancos de dados relacionais com **arquiteturas internas completamente diferentes**. A pergunta central é: *essa diferença de arquitetura afeta o desempenho? Em quais situações?*

Banco	Paradigma	Como funciona
PostgreSQL	Um processo por conexão	Cada cliente vira um processo separado no SO. Isolamento total, mas custo maior de memória.
MySQL (InnoDB)	Threads dentro de um processo	Um processo central atende todos os clientes via threads compartilhadas. Mais leve por conexão.
SQLite	Embarcado / sem servidor	Não existe "servidor". O código do banco roda dentro da própria aplicação e lê/escreve direto em um arquivo <code>.db</code> no disco. Zero overhead de rede.

2. Como testamos

Estrutura do benchmark

- **Dataset:** catálogo de ~123 mil álbuns musicais (RYM Top 5000), usado como dados reais para inserção e consulta.
- **Scripts:** um `script.py` por banco, todos com os mesmos parâmetros para garantir comparação justa.
- **Infraestrutura:** PostgreSQL e MySQL rodam em containers Docker; SQLite acessa um arquivo em volume local.

Parâmetros de execução

Parâmetro	Valor
Operações por rodada (INSERTs ou SELECTs)	100.000
Conexões simultâneas (cenários B e C)	100 workers
Operações por worker (B e C)	1.000
Repetições por experimento	10 (1 descartada como aquecimento, 9 consideradas)

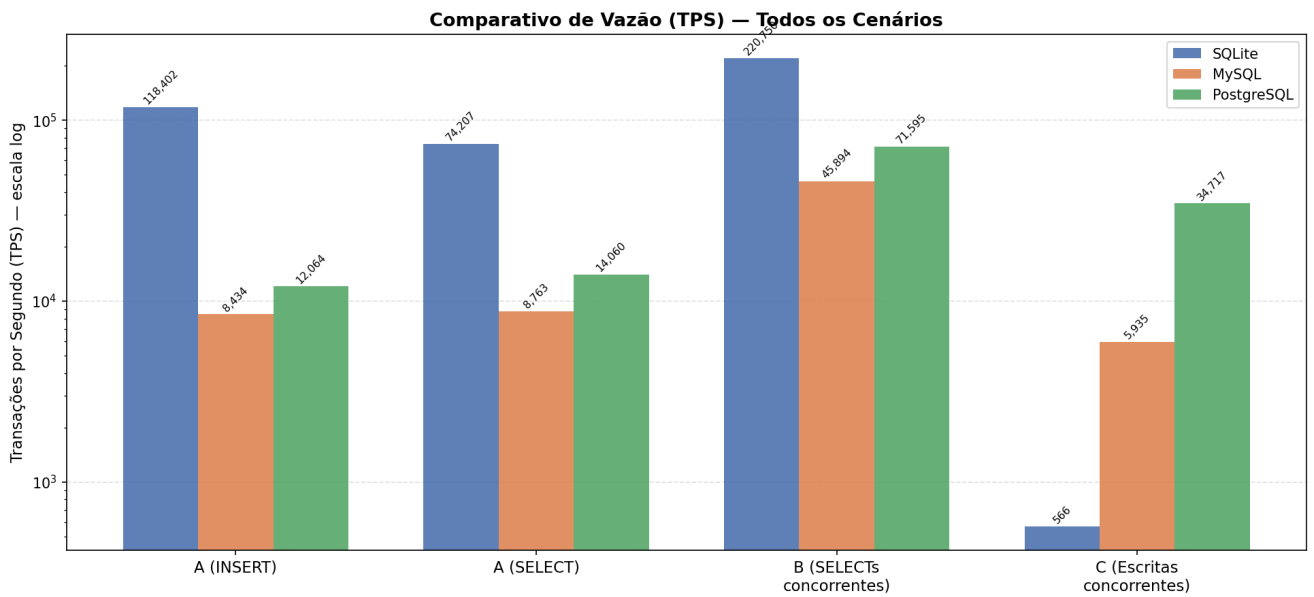
Ambiente

- **Máquina:** Ryzen 5 5600X, 32 GB RAM, Linux Ubuntu 25.10 x86_64
- **Rede:** comunicação via rede interna Docker (localhost) — sem latência de rede real entre os containers

3. Hipóteses esperadas

Cenário	O que esperávamos
A — Carga isolada	SQLite vence com folga: sem servidor, sem rede, acesso direto ao arquivo.
B — Leituras concorrentes	MySQL deveria se sair melhor que PostgreSQL (threads vs. processos). SQLite começaria a criar gargalos.
C — Escritas concorrentes	PostgreSQL lida melhor (MVCC robusto). SQLite entra em colapso com erros de "database is locked".

4. Resultados



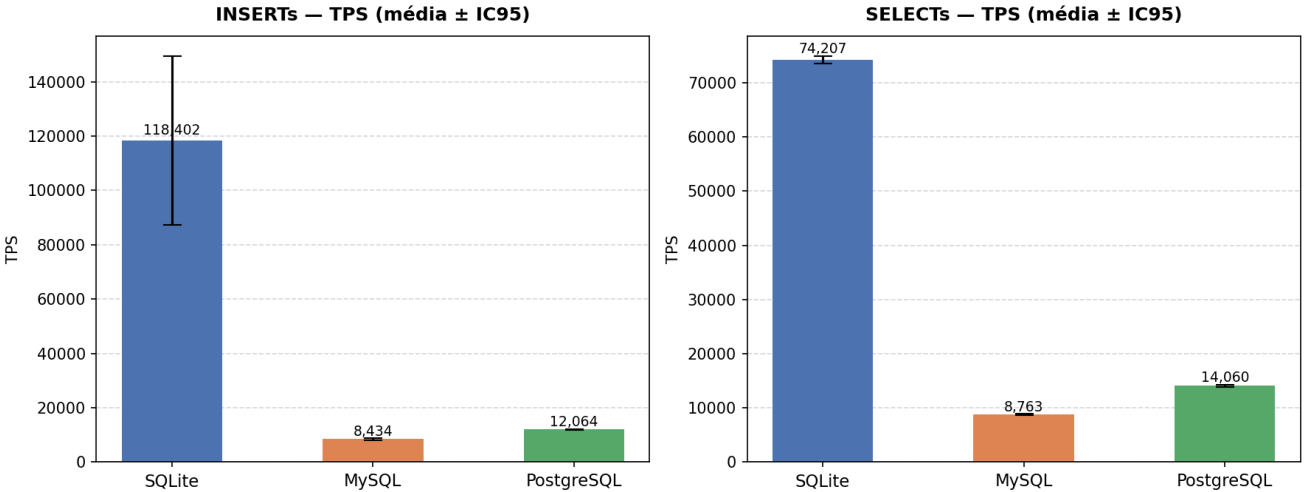
Leitura do gráfico acima: escala logarítmica. Cada grupo de barras é um cenário. Quanto mais alta a barra, mais transações por segundo — melhor desempenho.

Cenário A — Carga Isolada

O que é: uma única conexão inserindo 100k registros e depois lendo 100k registros. Sem concorrência.

O que observar: TPS (mais alto = mais rápido) e latência por operação (mais baixo = mais responsivo).

Cenário A — Carga Isolada (1 Conexão, 100k ops)



Resultados:

Banco	INSERT TPS	SELECT TPS	Lat. p99 INSERT	Lat. p99 SELECT
SQLite	118.402	74.207	0,01 ms	0,02 ms
PostgreSQL	12.064	14.060	0,11 ms	0,09 ms
MySQL	8.434	8.763	0,14 ms	0,15 ms

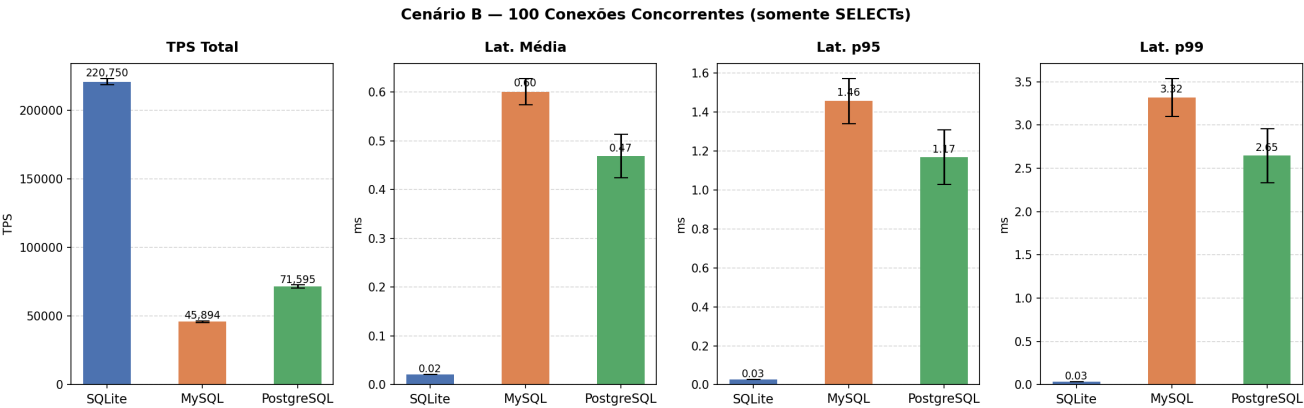
✓ Hipótese confirmada.

SQLite foi ~10x mais rápido que PostgreSQL e ~14x mais rápido que MySQL em INSERTs. A latência sub-milissegundo do SQLite confirma a vantagem do acesso direto ao arquivo: sem socket, sem protocolo de rede, sem processo servidor intermediário.

Cenário B — Leituras Concorrentes

O que é: 100 conexões simultâneas fazendo apenas SELECTs ao mesmo tempo.

O que observar: TPS total do sistema e latência — principalmente o p95 e p99, que mostram o quão "sofrido" foi para os clientes mais lentos.



Resultados:

Banco	TPS	Lat. média	Lat. p95	Lat. p99
SQLite	220.750	0,02 ms	0,03 ms	0,03 ms
PostgreSQL	71.595	0,47 ms	1,17 ms	2,65 ms
MySQL	45.894	0,60 ms	1,46 ms	3,32 ms

⚠ Hipótese parcialmente errada — resultado mais rico que o esperado.

Dois pontos divergiram:

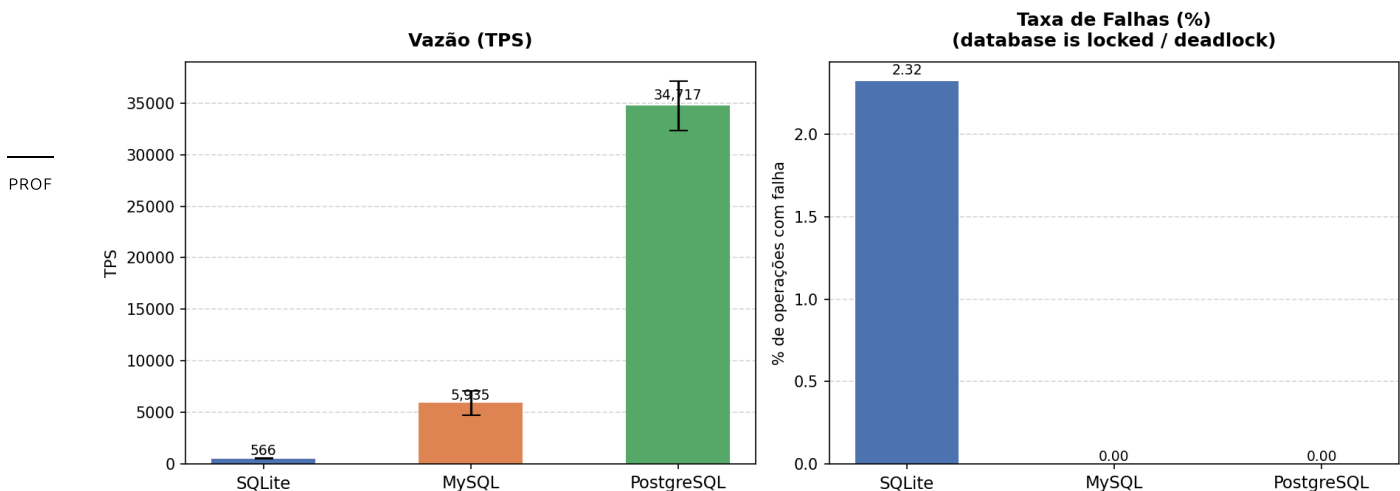
1. **SQLite não criou gargalo — dominou.** O SQLite permite múltiplos leitores simultâneos sem trava. Com 100 processos lendo o mesmo arquivo (que cabe inteiro na RAM), o sistema operacional serviu todos da memória sem overhead de rede. O paradigma embarcado foi ainda mais vantajoso aqui do que esperávamos.
2. **PostgreSQL bateu MySQL, não o contrário.** A hipótese era que threads (MySQL) seriam mais leves que processos (PostgreSQL) para muitas conexões. Isso é verdade para *memória*, mas o gargalo real aqui foi o overhead de protocolo de rede — que existe igualmente nos dois. A 100 conexões, a diferença de threads vs. processos não foi suficiente para o MySQL superar o PostgreSQL.

Cenário C — Escritas Concorrentes

O que é: 100 conexões simultâneas fazendo INSERTs e UPDATES ao mesmo tempo.

O que observar: TPS, taxa de falhas (operações que o banco recusou) e latência de escrita.

Cenário C — 100 Conexões Concorrentes (Escritas: 50% INSERT + 50% UPDATE)



Resultados:

Banco	TPS	Falhas	Lat. p99 INSERT	Lat. p99 UPDATE
PostgreSQL	34.717	0%	6 ms	6 ms
MySQL	5.935	0%	224 ms	343 ms

Banco	TPS	Falhas	Lat. p99 INSERT	Lat. p99 UPDATE
SQLite	566	2,32%	9 ms	8 ms

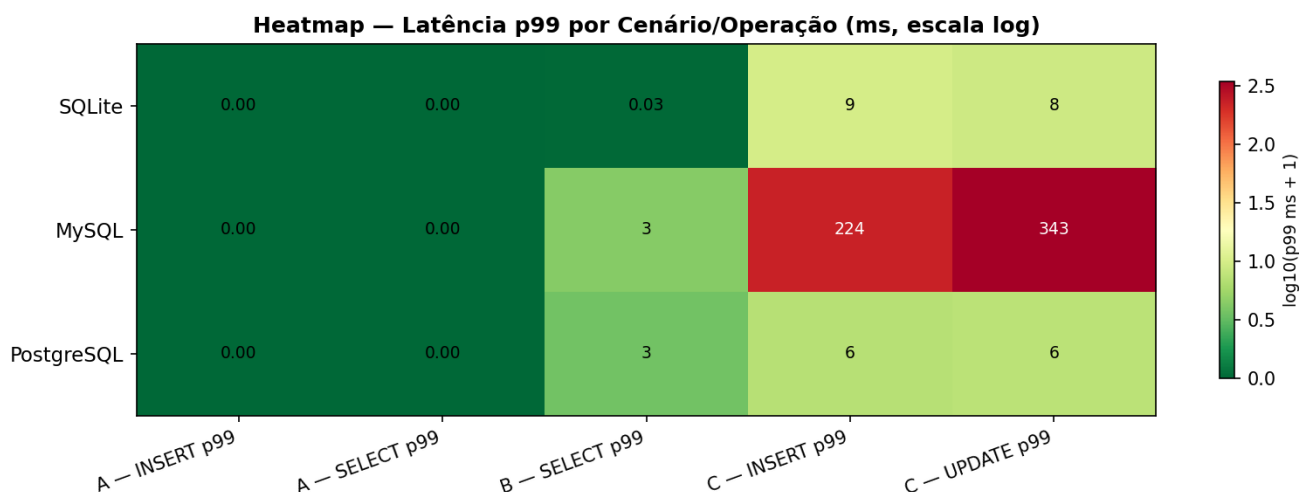
✓ **Hipótese confirmada — com surpresas no tamanho da diferença.**

- **SQLite travou como esperado.** Por padrão, o SQLite aceita apenas um escritor por vez. Com 100 tentando ao mesmo tempo, 99 ficam na fila. Se não conseguem o acesso em 5 segundos, falham. Resultado: 566 TPS e 2,32% das operações descartadas.
- **PostgreSQL ganhou de MySQL por ~6x**, não por uma margem pequena. Ambos têm MVCC e zeraram as falhas, mas o PostgreSQL foi muito superior em throughput. Um dos motivos: o MySQL por padrão força um **fsync** no disco a cada commit individual — e nesse benchmark cada operação tem seu próprio commit, tornando isso muito custoso.

5. Análises adicionais

Heatmap de pior caso (p99) por banco e cenário

O gráfico abaixo mostra, de uma vez só, onde cada banco teve seus piores momentos. **Verde = latência baixa. Vermelho = latência alta.**



O padrão visual confirma tudo que vimos:

- SQLite é verde escuro (excelente) nos cenários A e B, e vermelho intenso (3.000+ ms de p99) no C — o "colapso" do lock.
- PostgreSQL mantém latências baixas e estáveis em todos os cenários.
- MySQL é bom nos cenários A e B, mas tem p99 alto no C (224 ms e 343 ms), reflexo das filas de commit.

6. Conclusão

Os três paradigmas se comportaram exatamente como a teoria de banco de dados prevê — cada um com seu nicho claro:

Paradigma	Quando vence	Quando perde
SQLite (embarcado)	Usuário único ou leituras concorrentes locais: velocidade incomparável sem nenhum overhead de rede	Qualquer cenário com escritas concorrentes: o modelo de lock exclusivo serializa tudo
MySQL (threads)	Bom equilíbrio geral para leituras e escritas moderadas	A 100 conexões não superou o PostgreSQL; custo de commit individual é alto
PostgreSQL (processos)	Escritas concorrentes pesadas: MVCC e controle de transações robusto levam a 6x mais TPS que o MySQL no cenário C	Uso de memória por conexão mais alto (um processo por cliente)

A mensagem central: a escolha do SGBD não é sobre "qual é o mais rápido" em termos absolutos — é sobre qual paradigma se encaixa no padrão de acesso da aplicação. SQLite para uso local/embarcado, PostgreSQL para workloads de escrita concorrente, MySQL como opção intermediária para leituras em escala.